

SD3589CV100 NPU
AVP-QAT-Pytorch工具使用指南

文档版本　　V1.7.1

发布日期　　2024-12-17

前 言

概述

本文档主要介绍和说明AVP-QAT-Pytorch工具及使用方法等。

版本说明

与本文档相对应的产品版本如下。

产品名称	产品版本
SD3589C	V100
SD3589D	V100

读者对象

本文档(本指南)主要适用于以下工程师：

软件开发工程师、算法工程师

技术支持工程师

修订记录

修订记录包括每次文档更新的说明，最新版本的文档包含以前所有文档的版本更新内容。

修订日期	版本	修订说明
2023-11-17	V1.0.0	第一次版本发布
2023-12-28	V1.1.0	新增lsq量化算法
2024-02-05	V1.2.0	支持weight 4bit量化
2024-02-21	V1.2.2	解决resume恢复训练错误
2024-03-27	V1.3.0	新增PAT剪枝算法，yolov7量化示例

版权所有 © 上海云集互联有限公司

2024-04-03	V1.3.1	解决保存仅剪枝模型权重并载入
2024-06-07	V1.4.0	支持add/sub/mul/div/concat算子量化
2024-06-13	V1.4.1	解决模型剪枝错误
2024-08-05	V1.5.0	支持qat导出onnx模型转为板端模型
2024-09-24	V1.6.0	优化lsq算法初始化的赋值和耗时问题
2024-12-17	V1.7.1	修复yolov7示例

版权所有 © 上海云集互联有限公司

目 录

前 言 i
目 录 iii
插图目录 iv
1 简介 1
2 AVP-QAT-Pytorch工具使用 2
2.1 工具环境准备 2
2.2 分类网络量化示例 2
2.3 目标检测量化示例 5
2.4 剪枝结合量化示例 6
2.5 支持量化算子列表 10
附录 11
QAT相关限制 11
QAT建议 11

版权所有 © 上海云集互联有限公司

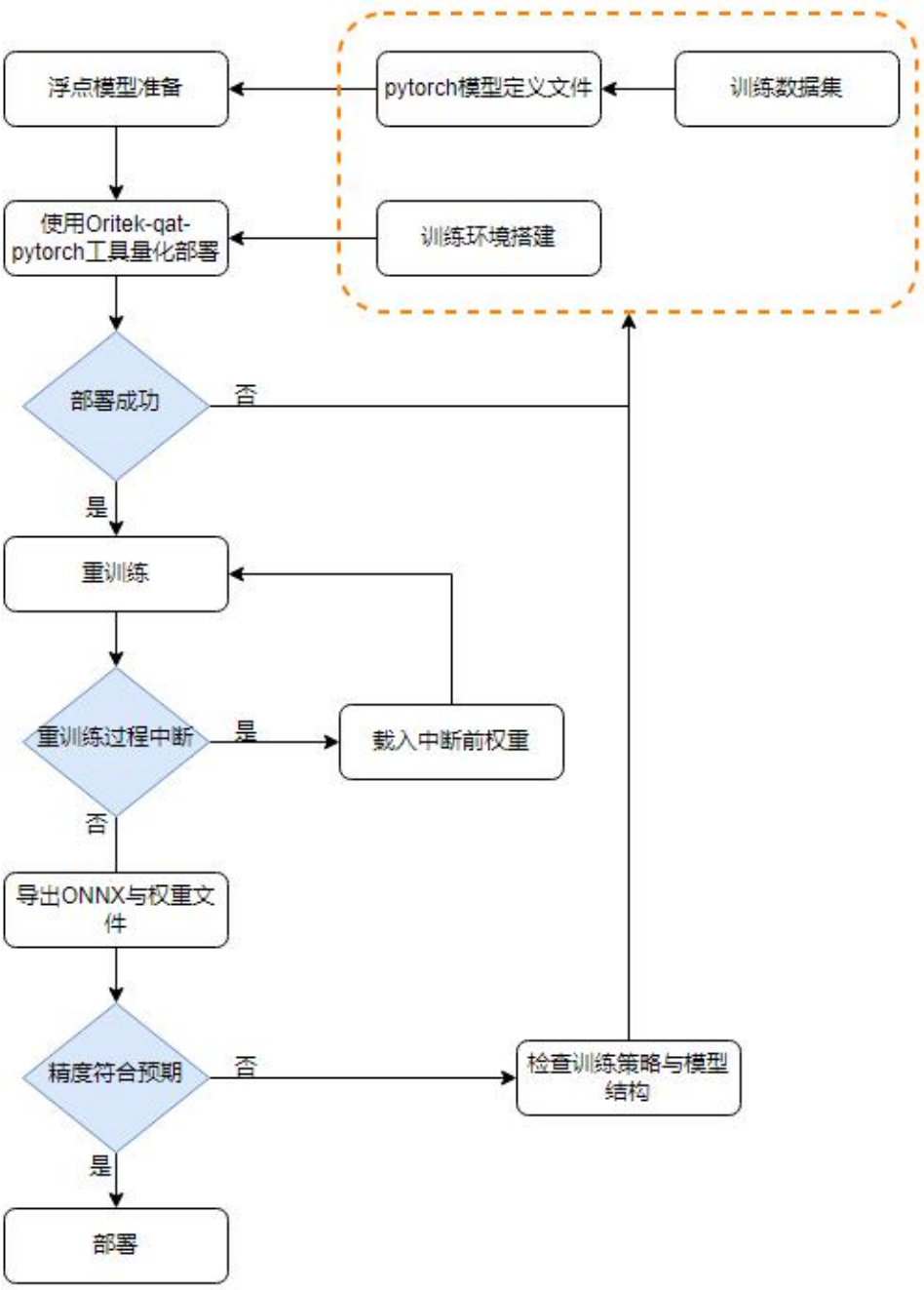
插图目录

图表 1-1 ORITEK-QAT-Pytorch工具主界面..... 1

版权所有 © 上海云集互联有限公司

1 简介

ORITEK-QAT-Pytorch 工具整体使用流程如图1-1所示。用户使用Python开源框架训练好模型，基于原始训练代码，导入ORITEK-QAT-Pytorch相关模块，调用初始化函数，既可以使用原始训练代码进行模型重训练，训练好的模型导出为ONNX，然后使用OMG工具进行模型转换为SD3589C的板端模型。



图表 1-1 ORITEK-QAT-Pytorch工具主界面

版权所有 © 上海云集互联有限公司

2 AVP-QAT-Pytorch 工具使用

2.1 工具环境准备

在Ubuntu20.04环境下，从发布包中获取oritek-qat-pytorch.whl包，直接安装到本地Python环境中，可以使用如下指令直接安装：

```
pip install -r requirements.txt
pip install oritek-qat-pytorch.whl
```

安装后即可直接使用。

requirements.txt列出需要的依赖包有以下：

```
torch==1.13.1
torchvision==0.11.1
torchaudio==0.13.1
onnx==1.14.0
onnxruntime-gpu==1.13.1
numpy==1.21.6
```

2.2 分类网络量化示例

在npu_toolchain/mct/sample目录中，包含一个AVP-QAT-Pytorch的demo工程，可以直接使用如下指令直接进行retrain：

1. 修改train.py的量化配置项

使用训练 command 前，需要根据采取的训练方案配置量化项 params，修改 train.py 中如下内容：

```
params = {“output”:xxx,"weight":xxx}
quantize_tool.initialize(**params)
```

（具体配置参数说明见 API 参考指南）

1）fake_quantizie 方案配置：

```
quantize_tool.initialize() # 激活：int8 非对称 per-tensor 量化；权重：int8 对称 per-channel 量化
```

2）lsq per-tensor 方案配置：

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True},
          "weight":{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True}}
quantize_tool.initialize(**params)
```

3）lsq per-channel 方案配置：

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True},
          "weight":{"num_bits":8, "quant_alg":"lsq", "per_channel":True, "axis":0, "unsigned":False, "symmetric":True}}
quantize_tool.initialize(**params)
```


版权所有 © 上海云集互联有限公司

4）fake_quantizie 方案配置（4bit）：

```
params = {“output”:{"num_bits":8, "quant_alg":"fake_quantization", "per_channel":False, "axis":0, "unsigned":False, "symmetric":False},
          "weight":{"num_bits":4, "quant_alg":"fake_quantization", "per_channel":True, "axis":0, "unsigned":False, "symmetric":True}}
quantize_tool.initialize(**params)
```

5）lsq per-channel 方案（4bit）配置：

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True},
          "weight":{"num_bits":4, "quant_alg":"lsq", "per_channel":True, "axis":0, "unsigned":False, "symmetric":True}}
```

2. 训练命令

resnet18:

(1) fake_quantization 方案:

修改 train.py 配置项(第 231 行)：

```
quantize_tool.initialize()
python train.py --model resnet18 --output-dir output/resnet18 --epochs 110 --batch-size 256 --weights DEFAULT \
--wd 0.0001 --lr 0.001 --lr-step-size 40    --quant    --data-path xxx
```

mobilenet_v3_large:

(1) 训练采取 lsq per-tensor 方案:

修改 train.py 配置项(第 231 行)：

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True},
          "weight":{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True}}
quantize_tool.initialize(**params)
python train.py --model mobilenet_v3_large --output-dir output/mobilenet_v3_large/ --epochs 110 --batch-size 192 \
--weights DEFAULT --wd 0.00001 --lr 0.0065 --lr-step-size 40 --quant    --mixup-alpha 0.2 --cutmix-alpha 0.2 --data-path xxx --set-parameter-alpha
```

(2) 训练采取 lsq per-channel 方案:

修改 train.py 配置项(第 231 行)：

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True},
          "weight":{"num_bits":8, "quant_alg":"lsq", "per_channel":True, "axis":0, "unsigned":False, "symmetric":True}}
quantize_tool.initialize(**params)
python train.py --model mobilenet_v3_large --output-dir output/mobilenet_v3_large/ --epochs 110 --batch-size 192 \
```


版权所有 © 上海云集互联有限公司

--weights DEFAULT --wd 0.00001 --lr 0.005 --lr-step-size 30 --quant --data-path xxxx --set-parameter-alpha

mobilenet_v3_small:

推荐使用 lsq per-channel 方案

(1)训练采取 lsq per-channel 方案

修改 train.py 配置项(第 231 行)：

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True},
                "weight":{"num_bits":8, "quant_alg":"lsq", "per_channel":True, "axis":0, "unsigned":False, "symmetric":True}}
quantize_tool.initialize(**params)
```

```
python train.py --model mobilenet_v3_small --output-dir output/mobilenet_v3_small --epochs 110 --batch-size 768 \
--weights DEFAULT --wd 0.00001 --lr 0.006 --lr-step-size 50 --quant --data-path xxx --set-parameter-alpha
```

resnet18 4bit：

(1) 训练采取 lsq per-channel 方案（4bit）

修改 train.py 配置项(第 231 行)：

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False, "symmetric":True},
                "weight":{"num_bits":4, "quant_alg":"lsq", "per_channel":True, "axis":0, "unsigned":False, "symmetric":True}}
quantize_tool.initialize(**params)
```

```
python train.py --model resnet18 --output-dir output/resnet18 --epochs 60 --batch-size 256 --weights DEFAULT \
--wd 0.0001 --lr 0.001 --lr-step-size 15 --quant --data-path xxx --set-parameter-alpha
```

命令行参数含义：

1、model：模型名称
2、output-dir：输出文件夹路径
3、epochs：训练轮次
4、batch-size：批大小
5、weights：权重路径。torchvision 训练模型则是“DEFAULT”
6、wd：权重衰减率
7、lr：学习率
8、lr-step-size：学习率衰减步长
9、quant：是否开启量化。默认为 false
10、data-path：数据集路径
11、set-parameter-alpha：是否单独设置量化参数 alpha 的学习率。选择 lsq 量化算法才需开启，默认为 false。

版权所有 © 上海云集互联有限公司

AVP-QAT-Pytorch 使用步骤（具体接口说明见API参考指南）：

1、调用 oritek-qat.initialize 进行初始化，传入量化配置项。
2、调用 oritek-qat.initialize_quantize 进行量化使能。
3、调用原始代码中的训练代码。
4、调用 oritek-qat.export_onnx 将 pytorch 模型导出为 onnx 模型

伪代码：

```
1.开启量化
from oritek_qat import quantize_tool
quantize_tool.initialize()
model = xxx # 注意：cpu 模型
2.初始化量化阈值
for image, _ in data_loader:
    batch_data = image
    break
# batch_data 是训练集中的一个随机批次数据集，会影响重训的收敛效果，batch_data 的批大小建议设置为256
# 模型多输入则以 tuple 类型传入，例如 batch_data = (input1, input2, ..., inputn)
model = quantize_tool.create_quant_model(model, batch_data)
model.to(device)
3.(可选)若量化算法为 lsq，重设量化参数 alpha 的 lr
parameters = quantize_tool.set_lsq_algo_parameter(model, parameter_list=None)
4.训练与评估过程
for epoch in range(start_epoch, epochs):
    train_one_epoch(model, ..., scaler)
    quantize_tool.disable_observer(model) #注意：evaluate 过程关闭量化收集器
    evaluate(model, criterion, data_loader_test, device=device)
    quantize_tool.enable_observer(model)
5.导出
x = torch.randn(1, 3, args.val_crop_size, args.val_crop_size).to(device)
quantize_tool.export_onnx(model_without_ddp, x, os.path.join(args.output_dir, "final_retrain.onnx"))
```

2.3 目标检测量化示例

在npu_toolchain/mct/sample/detection目录中，包含目标检测模型的量化demo工程，以yolov7为例：

1. 环境与文件准备

```
环境准备
conda activate your_env_name
pip install -r requirements.txt
pip install oritek-qat-pytorch-x.x.x.whl
```


版权所有 © 上海云集互联有限公司

文件准备

在 yolov7 文件夹下创建 weight 文件夹

在官网下载 yolov7.pt 到 yolov7/weight 文件夹（下载链接：

https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7.pt）

修改 coco.yaml 中的 coco 数据集路径

2. 训练命令

```
python -m torch.distributed.launch --nproc_per_node 2 --master_port 9528   yolov7_qat.py \
--device 0,1 \
--sync-bn \
--batch-size 32 \
--img 640 640 \
--data data/coco.yaml \
--cfg cfg/deploy/yolov7.yaml \
--weights "weights/yolov7.pt" \
--hyp data/hyp.scratch.p5.yaml \
--name yolov7 \
--epochs 50 \
--quant
```

命令行参数含义：

1、device：指定 GPU 卡号

2、sync-bn：是否使用跨卡同步 BN,在 DDP 模式使用

3、batch-size：批大小

4、img：输入图像的宽高

5、data：数据集 YAML 路径

6、cfg：模型结构文件路径

7、weights：模型权重路径

8、hyp：训练超参数文件路径

9、name：模型名称

10、epochs：训练轮次

11、quant：启用量化

2.4 剪枝结合量化示例

在npu_toolchain/mct/sample目录中，包含一个支持剪枝并量化训练demo工程，可以直接使用如下指令直接进行retrain：

1.训练 resnet18

修改 train_prune 配置项(第 309 行)：

版权所有 © 上海云集互联有限公司

```
params = {“output”:{"num_bits":8, "quant_alg":"lsq", "per_channel":False, "axis":0, "unsigned":False,
"symmetric":True},
            "weight":{"num_bits":8, "quant_alg":"lsq", "per_channel":True, "axis":0, "unsigned":False,
"symmetric":True}}
quantize_tool.initialize(**params)
python train_prune.py --model resnet18 --output-dir output/pat/resnet18 --epochs 60 --batch-size 256 --weights
DEFAULT --wd 0.0001 --lr 0.001 --lr-step-size 40 --data-path xxx --sparsity 0.5    --pruner level_unstructured --
lamp-type lamp
```

剪枝参数含义：

1、sparsity：权重总体剪枝率；取值范围为[0,1)浮点数，默认设置为 0.5；

2、pruner：剪枝器。默认设置为'level_unstructured'，有三种可选模式如下：

'level_unstructured'：对每个权重 tensor 执行 L1-norm 的稀疏；

'level_unstructured_finer'：对每个权重 tensor 的每个 output 通道执行 L1-norm 的稀疏（即量化中 per-channel 维度）；

'sparse_model_processer'：对已稀疏模型执行 finetune，可根据加载的稀疏模型生成 weight 的 sparse mask，并计算对应的稀疏度（须载入已剪枝模型），便于直接 finetune 训练或者 qat 重训

3、lamp-type：稀疏率计算算法。默认设置为'lamp'，有五种可选算法如下（具体见使用文档）：

'None'：均匀稀疏，每层权重的稀疏率都为 sparsity；

'lamp'：非均匀稀疏，基于 LAMP 算法（Layer-Adaptive Magnitude-based Pruning）；

'l1norm'：非均匀稀疏，基于单个权重值的 L2-norm 相对于整个权重矩阵的 L2-norm 公式计算；

'l2norm'：非均匀稀疏，根据每个权重值的 L1-norm（绝对值）计算；

'l1norm_local'：非均匀稀疏，基于单个权重值相对于整个权重矩阵的 L1-norm 公式计算；

4、checkpoint_path: 已剪枝权重的路径

如果—pruner 设置为'sparse_model_processer'，需要加载已剪枝模型，使用—checkpoint_path 配置已剪枝模型的权重

仅剪枝使用步骤：

```
1. 开启量化
from oritek_qat import prune_tool
from oritek_qat import quantize_tool
quantize_tool.initialize()
model = xxx
for image, _ in data_loader:
batch_data = image
    break
# batch_data 是训练集中的一个随机批次数据集，会影响重训的收敛效果，batch_data 的批大小建议设置为
256
#  模型多输入则以 tuple 类型传入，例如 batch_data = (input1, input2, ..., inputn)
    model = quantize_tool.create_quant_model(model, batch_data)
```


版权所有 © 上海云集互联有限公司

```
quantize_tool.disable_quantize(model) #注：仅剪枝/剪枝后量化，都在剪枝前需要关闭量化；量化后剪枝不需要
2.计算稀疏率
prune_tool.init_pruner_and_cal_sparsities(model, args.sparsity, args.pruner, args.lamp_type)
3.剪枝
x = torch.randn(1, 3, args.val_crop_size, args.val_crop_size)
prune_tool.execute_pruning(model, x)
4.保存剪枝权重
if (not args.distributed) or torch.distributed.get_rank() == 0:
    model_path = os.path.join(args.output_dir, '{}_pruned.pth'.format(args.model))
    torch.save(model.stat_dict(), model_path)
model.to(device)

5.(可选)浮点 finetune
optimizer = xxx # set your optimizer
prune_tool.initializing_optimizer_for_pruning(optimizer)
for epoch in range（start_epoch, epochs）:
    train_one_epoch(model, ..., scaler)
```

先剪枝后量化使用步骤：

```
1.开启量化
from oritek_qat import quantize_tool
from oritek_qat import prune_tool
quantize_tool.initialize()
model = xxx
quantize_tool.disable_quantize(model) #注：仅剪枝/剪枝后量化，都在剪枝前需要关闭量化；量化后剪枝不需要
2.初始化量化阈值
for image, _ in data_loader:
    batch_data = image
    break
# batch_data 是训练集中的一个随机批次数据集，会影响重训的收敛效果，batch_data 的批大小建议设置为256
#  模型多输入则以 tuple 类型传入，例如 batch_data = (input1, input2, ..., inputn)
model = quantize_tool.create_quant_model(model, batch_data)
3.计算稀疏率
prune_tool.init_pruner_and_cal_sparsities(model, args.sparsity, args.pruner, args.lamp_type)
4.剪枝
x = torch.randn(1, 3, args.val_crop_size, args.val_crop_size)
prune_tool.execute_pruning(model, x)
5.保存剪枝权重并导出 ONNX
if (not args.distributed) or torch.distributed.get_rank() == 0:
```


版权所有 © 上海云集互联有限公司

```
        model_path = os.path.join(args.output_dir, '{}_pruned.pth'.format(args.model))
        torch.save(model.state_dict(), model_path)
model.to(device)

==========================================
6.更新 Optimizer
pruner_excutor.initializing_optimizer_for_pruning(optimizer)
7.训练与评估过程
for epoch in range（start_epoch, epochs）:
        train_one_epoch(model, ..., scaler)
        quantize_tool.disable_observer(model) #注意：evaluate 过程关闭量化收集器
        evaluate(model, criterion, data_loader_test, device=device)
        quantize_tool.enable_observer(model)
8.导出
x = torch.randn(1, 3, args.val_crop_size, args.val_crop_size).to(device)
quantize_tool.export_onnx(model_without_ddp, x, os.path.join(args.output_dir, "final_retrain.onnx"))
```

先量化后剪枝使用步骤：

```
1.开启量化
from oritek_qat import quantize_tool, prune_tool
quantize_tool.initialize()
model = xxx
2.初始化量化阈值
for image, _ in data_loader:
        batch_data = image
        break
# batch_data 是训练集中的一个随机批次数据集，会影响重训的收敛效果，batch_data 的批大小建议设置为
256
#  模型多输入则以 tuple 类型传入，例如 batch_data = (input1, input2, ..., inputn)
model = quantize_tool.create_quant_model(model, batch_data)
model.to(device)
3.训练与评估过程
for epoch in range（start_epoch, epochs）:
        train_one_epoch(model, ..., scaler)
        quantize_tool.disable_observer(model) #注意：evaluate 过程关闭量化收集器
        evaluate(model, criterion, data_loader_test, device=device)
        quantize_tool.enable_observer(model)
==========================================
4.计算稀疏率
prune_tool.init_pruner_and_cal_sparsities(model, args.sparsity, args.pruner, args.lamp_type)
5.剪枝
x = torch.randn(1, 3, args.val_crop_size, args.val_crop_size).to(device)
```


版权所有 © 上海云集互联有限公司

```
prune_tool.execute_pruning(model, x)
6.导出
x = torch.randn(1, 3, args.val_crop_size, args.val_crop_size).to(device)
quantize_tool.export_onnx(model_without_ddp, x, os.path.join(args.output_dir, "final_retrain.onnx"))
```

2.5 支持量化算子列表

算子	Pytorch module
Conv	torch.nn.Conv1d
	torch.nn.Conv2d
	toch.nn.Conv3d
ConvTranspose	torch.nn.ConvTranspose1d
	torch.nn.ConvTranspose2d
	torch.nn.ConvTranspose3d
fc	torch.nn.Linear
BatchNorm	torch.nn.BatchNorm1d
	torch.nn.BatchNorm2d
	torch.nn.BatchNorm3d
activation	torch.nn.ReLU
	torch.nn.ReLU6
	torch.nn.Sigmoid
	torch.nn.SiLU
	torch.nn.SELU
	torch.nn.GELU
	torch.nn.LeakyReLU
	torch.PReLU
pooling	torch.nn.MaxPool1d
	torch.nn.MaxPool2d
	torch.nn.MaxPool3d
	torch.nn.AvgPool1d
	torch.nn.AvgPool2d
	torch.nn.AvgPool3d
add	torch.add
	.add
	operator.add
sub	torch.sub
	.sub
	operator.sub

版权所有 © 上海云集互联有限公司

mul	torch.mul
	.mul
	operator.mul
div	torch.div
	.div
	operator.div
concat	torch.concat
	torch.cat
	torch.concatenate

附录

QAT 相关限制

1. Conv Gemm的weight只能是对称量化，激活函数只支持per-tensor量化。

2. MatMul的输入都必须是对称量化。

3. 权重支持4bit量化，但是格式必须是 Round->Clip->Mul。

4. 量化系数范围需要限制在FP16取值范围内，否则工具链中会将超过FP16范围的数据强制转为FP16边界值。

QAT 建议

1. 如果融合BN之后再进行训练，容易产生无法收敛或收敛效果较差的问题。但是带着BN训练，又会导致BN无法和Conv融合的问题。这里建议参考pytorch官方提出的QAT融合方式。

2. 训练建议：

初始尝试建议：

方向	建议
训练配置项	算法： 常规模型：建议首选fake quantization算法，若效果不好则替换为lsq；空洞卷积模型等复杂模型：首选lsq算法 per-tensor/per-channel： 复杂模型per-channel精度略高点但训练耗时长
训练参数	首先尝试浮点模型训练的参数，但是注意alpha参数要单独设置

版权所有 © 上海云集互联有限公司

低比特训练（W4A8）	建议使用freeze bn策略 （在第五个epoch结束冻结BN算子）
剪枝+量化训练	建议lsq算法

初次训练的参数建议使用浮点模型训练的参数，根据结果调整训练参数，建议如下：

参数/训练策略	建议设置	备注
lr	从浮点模型训练的lr开始尝试	1.连续训练三个epoch，确定初始精度走向 2.调大或减小20%~30%学习率，确定最大的初始精度
epochs	110	训练2/3epochs到结束epochs精度是否稳定在一定范围，如果稳定则不需要增加epochs
lr-step-size	30~40	30基本足够
weight_decay	0.00001	
Bn融合方式	谷歌白皮书建议方式	
alpha设置lr和wd	weight: 0.001 activation: 0.002 weight_decay:均为0.00001	1.建议开启，联合bn融合方式可略减小震荡 2.若不开启，初始lr设置0.01再做调试
数据增广	默认无	可尝试cut-mix=0.2；fix-mix=0.2是否提升精度

版权所有 © 上海云集互联有限公司